

iSCSI Diagnostics & Error Reporting Extension

iSCSI Diagnostics & Error Reporting Extension

This document explains the architecture and implementation reasoning behind the new error reporting and diagnostics system added to the Python CLI metrics collector.

The goal of this feature is to evolve the CLI from a simple metrics collector into a practical infrastructure diagnostics tool capable of detecting:

- iSCSI failures
- storage issues
- network problems
- kernel-level I/O failures
- initiator/target connectivity issues

Table of Contents

- Overview
- Step 1 — NodeErrorReport Dataclass
- Step 2 — ERROR_PATTERNS
- Step 3 — collect_recent_logs()
- Step 4 — scan_logs_for_errors()
- Step 5 — Initiator Node Support
- Step 6 — collect_node_diagnostics()
- Architectural Design Decisions
- Future Improvements

Overview

Originally, the CLI only collected:

- iSCSI target information
- LUN statistics
- read metrics

- command counters

The new diagnostics system introduces:

- log collection
- infrastructure error detection
- structured diagnostics
- subsystem categorization
- severity classification

This allows the CLI to provide operational insights instead of only raw metrics.

Step 1 — NodeErrorReport Dataclass

Purpose

The existing errors dictionary in the project only represented:

- Collector/runtime failures

Examples:

- pdsh failed
- file read failed
- command execution failed

However, the new requirement introduces:

- Actual infrastructure failures

Examples:

- iSCSI connection loss
- I/O errors
- storage disconnects
- network timeouts

These are fundamentally different concepts.

Implementation

```
from dataclasses import dataclass
```

```
@dataclass
class NodeErrorReport:
    node: str
    source: str
    severity: str
    message: str
```

```
NodeErrorReport(
    node="worker-1",
    source="iscsi",
    severity="critical",
    message="session recovery failed"
)
```

```
ERROR_PATTERNS = [
    {
        "pattern": r"connection.*lost",
        "severity": "warning",
        "source": "iscsi",
    },
    {
        "pattern": r"session.*failure",
        "severity": "warning",
        "source": "iscsi",
    },
    {
        "pattern": r"timed out",
        "severity": "warning",
        "source": "network",
    },
    {
        "pattern": r"blocked for more than",
        "severity": "critical",
        "source": "kernel",
    },
    {
        "pattern": r"I/O error",
        "severity": "critical",
        "source": "storage",
    },
    {
        "pattern": r"rejecting I/O to offline device",
        "severity": "critical",
        "source": "storage",
    },
]
```

```

def collect_recent_logs(
    node: str,
    lines: int = 200,
) -> Tuple[str, Optional[str]]:
    commands = [
        f"journalctl -n {lines} --no-pager 2>/dev/null",
        f"tail -n {lines} /var/log/messages 2>/dev/null",
        f"tail -n {lines} /var/log/syslog 2>/dev/null",
        f"dmesg | tail -n {lines}",
    ]

    for command in commands:
        output, error = run_pdsh_text(
            f'pdsh -w {node} "{command}"'
        )

        if output:
            return output, None

    return "", f"{node}: unable to collect logs"

```

```

def scan_logs_for_errors(
    node: str,
    log_text: str,
) -> List[NodeErrorReport]:
    findings: List[NodeErrorReport] = []

    for line in log_text.splitlines():
        lowered = line.lower()

        for entry in ERROR_PATTERNS:
            if re.search(entry["pattern"], lowered, re.IGNORECASE):
                findings.append(
                    NodeErrorReport(
                        node=node,
                        source=entry["source"],
                        severity=entry["severity"],
                        message=line.strip(),
                    )
                )

    return findings

```

```

DEFAULT_INITIATOR_SELECTOR = "iscsi-initiator=true"

```

```

parser.add_argument(
    "--initiator-selector",
    default=DEFAULT_INITIATOR_SELECTOR,
    help="Kubernetes label selector used to discover iSCSI initiator nodes",
)

```

```
def collect_node_diagnostics(
    node: str,
) -> Tuple[List[NodeErrorReport], List[str]]:
    errors: List[str] = []

    logs, log_error = collect_recent_logs(node)

    if log_error:
        return [], [log_error]

    findings = scan_logs_for_errors(node, logs)

    return findings, errors
```

Why This Structure Matters

This dataclass defines the diagnostic model used throughout the system.

Instead of storing raw strings:

```
all_errors.append(line)
```

we now store structured diagnostics.

Benefits of using @dataclass:

- cleaner code
- structured access
- easier serialization
- maintainability
- extensibility

Why Regex Is Used

System logs vary across:

- Linux distributions
- kernel versions
- storage stacks

Regex provides flexible matching and avoids fragile exact string matching.

Why Centralized Rules Matter

Keeping all detection rules in one location improves:

- maintainability
- scalability
- readability
- future expansion

Why Use Fallback Collection

Different Linux systems expose logs differently.

This fallback mechanism improves portability and compatibility.

Why Structured Parsing Matters

Without structured parsing:

- logs remain noisy
- manual inspection becomes harder
- automation becomes difficult

Structured diagnostics support:

- filtering
- reporting
- alerting
- severity grouping

Why Separate Selectors

Target nodes and initiator nodes serve different purposes.

Separate selectors improve flexibility and support mixed environments.

Why This Abstraction Is Useful

`collect_node_diagnostics()` creates a reusable diagnostics workflow that combines:

- log collection
- parsing
- rule matching

- diagnostics generation

This keeps the main collection flow cleaner and easier to maintain.